

JAVA PROGRAMMING LAB

Experiment No: Week 06

Date: 16/09/26

Title: Finding the Longest Substring Without Repeating Characters

1. Objective / Aim

To write and execute a Java program that finds the longest substring of a given string without repeating characters and displays both the substring and its length.

2. System Requirements

Hardware Requirements	Software Requirements
• Computer/Laptop • Minimum 4 GB RAM • Minimum 1 GHz processor • Keyboard and monitor	• Operating System: Windows / Linux / macOS • JDK 17 or JDK 21 • IDE: Eclipse / IntelliJ IDEA / NetBeans / VS Code • Command Prompt / Terminal

3. Algorithm / Logic Description

Step 1: Start the program.

Step 2: Create a Scanner object and read a string from the user.

Step 3: Initialize longest as an empty string and start as 0.

Step 4: Traverse the string character by character using a for loop.

Step 5: For every character, check whether it is already present in the current substring.

Step 6: If a repeated character is found, move the start position to the character after the repeated character.

Step 7: Create the current substring using substring(start, i + 1).

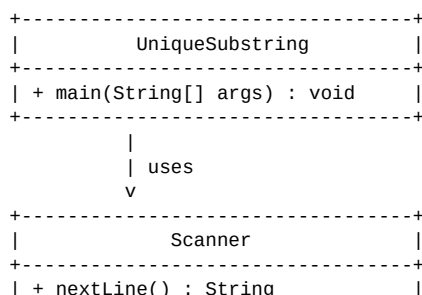
Step 8: Compare its length with the longest substring found so far.

Step 9: If it is longer, store it in longest.

Step 10: Display the longest substring and its length.

Step 11: Close the Scanner and stop the program.

4. UML Class Diagram



```
| + close() : void |
+-----+

```

The UniqueSubstring class contains the main() method. The program uses the Scanner class to read input from the user.

5. Source Code

```
import java.util.Scanner;

// Main class
public class UniqueSubstring {

    public static void main(String[] args) {

        // Create Scanner object to take input from the user
        Scanner sc = new Scanner(System.in);

        // Ask the user to enter a string
        System.out.print("Enter a string: ");
        String str = sc.nextLine();

        // Variable to store the longest substring
        String longest = "";

        // Starting position of the current substring
        int start = 0;

        // Loop through each character of the string
        for (int i = 0; i < str.length(); i++) {

            // Check for repeated characters
            for (int j = start; j < i; j++) {

                // Compare the current character with previous characters
                if (str.charAt(j) == str.charAt(i)) {

                    // Move the starting position after the repeated character
                    start = j + 1;
                    break;
                }
            }

            // Get the current substring without repeating characters
            String current = str.substring(start, i + 1);

            // Check if the current substring is longer
            if (current.length() > longest.length()) {

                // Store the current substring as the longest substring
                longest = current;
            }
        }

        // Display the longest substring
        System.out.println("Longest substring without repeating characters: "
            + longest);

        // Display the length of the longest substring
        System.out.println("Length: " + longest.length());

        // Close the Scanner object
        sc.close();
    }
}
```

6. Compilation and Execution Commands

Step 1: Save the program as:
UniqueSubstring.java

Step 2: Open Command Prompt / Terminal in the program folder.

Step 3: Compile the program:
javac UniqueSubstring.java

Step 4: Execute the program:
java UniqueSubstring

7. Output

Sample Test Case:

```
Enter a string: abcabcb  
Longest substring without repeating characters: abc  
Length: 3
```

Result / Completion: The Java program was successfully compiled and executed. It finds and displays the longest substring without repeating characters and its length.